

AI Agent / Local RAG Projesi — Teknik Geliştirme Raporu

1. Projenin amacı

Bu çalışmanın temel amacı, ücretli bir LLM API'sine bağımlı olmadan çalışan, tamamen lokal ve açık kaynak bileşenlerden oluşan bir **Retrieval-Augmented Generation (RAG)** sistemi geliştirmektir.

Başlangıçta hedef sadece bir chatbot oluşturmak değildi. Sistem zamanla daha mühendislik odaklı bir yapıya dönüştü. Nihai amaç, kullanıcının sorduğu soruya cevap verirken hem doğru doküman parçalarını bulabilen hem üretilen cevabın gerçekten dokümana dayandığını kontrol edebilen hem de yeterli bilgi bulunmadığında cevap vermeyi reddedebilen güvenilir bir pipeline oluşturmaktır.

Projenin temel prensibi şu oldu:

Modelin bir cevap üretebilmesi tek başına yeterli değildir. Üretilen bilgi kaynak tarafından desteklenmeli ve desteklenmiyorsa sistem cevap vermemelidir.

2. Geliştirme ortamı

Proje macOS üzerinde VS Code ve terminal kullanılarak geliştirildi.

Proje klasörü:

/Users/berketuylek/Desktop/ai-agent-project

Python için ayrı bir virtual environment oluşturuldu:

.venv

Kullanılan temel teknolojiler şunlardı:

Bileşen	Kullanılan teknoloji
Programlama dili	Python 3.12
LLM	Qwen/Qwen2.5-0.5B-Instruct
Embedding modeli	all-MiniLM-L6-v2
Semantic search	scikit-learn NearestNeighbors
Similarity metric	Cosine similarity
NLI verifier	cross-encoder/nli-deberta-v3-base
Transformer altyapısı	Hugging Face Transformers
Deep Learning	PyTorch

Bileşen	Kullanılan teknoloji
Embedding altyapısı	sentence-transformers
Veri yapıları	NumPy / JSON
Version control	Git
Remote repository	GitHub

Başlangıçta FAISS kullanılması değerlendirildi. Ancak lokal ortamda FAISS kurulumu sırasında swig bağımlılığı nedeniyle build problemi yaşandı. Bunun yerine bu ölçekte fazlasıyla yeterli olan:

```
NearestNeighbors(  
    metric="cosine",  
    algorithm="brute"  
)
```

kullanıldı.

Bu karar sistemin ücretsiz ve lokal çalışması hedefiyle de uyumluydu.

3. Proje öncesinde öğrenilen temel kavramlar

RAG geliştirilmeden önce transformer tabanlı sistemlerin mantığını anlamaya yönelik deneyler gerçekleştirildi.

İlk aşamada tokenizer mantığı incelendi. Metnin doğrudan modele verilmediği, önce tokenlara ve daha sonra token ID'lerine dönüştürüldüğü görüldü.

Ardından embedding kavramı üzerinde çalışıldı. DistilBERT kullanılarak contextual embedding üretimi denendi. Daha sonra semantic similarity uygulamaları için klasik BERT embeddinglerinin yerine sentence-transformers kullanılmasının daha uygun olduğu görüldü.

Bu aşamada iki cümle arasındaki benzerlik cosine similarity ile ölçüldü.

Mantık:

```
Sentence  
↓  
Tokenizer  
↓  
Transformer  
↓  
Embedding Vector  
↓  
Cosine Similarity
```

Daha sonra semantic retrieval geliştirildi ve bir sorgunun en yakın metin parçalarının bulunması sağlandı.

Bu çalışma RAG pipeline'ının retrieval katmanının temelini oluşturdu.

4. RAG bilgi kaynağı

İlk kontrollü deneyler için küçük fakat farklı politika türleri içeren bir bilgi kaynağı oluşturuldu.

Doküman şu bilgileri içeriyordu:

Customers can return products within 30 days after delivery.

The product must be unused and in its original packaging.

Refunds are processed within 5 business days after the returned product is received.

Standard shipping usually takes between 3 and 5 business days.

Express shipping is available and normally takes 1 to 2 business days.

Shipping times may be longer during holidays.

All electronic products include a two-year warranty.

The warranty covers manufacturing defects but does not cover accidental damage or damage caused by improper use.

Premium members receive free standard shipping on all orders.

They also receive early access to discounts and special promotions.

Premium membership can be cancelled at any time.

Bu küçük bilgi kaynağı özellikle seçildi çünkü aynı doküman içerisinde:

- sayısal bilgiler,
- süreler,
- pozitif kurallar,
- negatif kurallar,
- üyelik avantajları,
- garanti koşulları,
- return-policy şartları

bulunuyordu.

Bu sayede semantic retrieval, hallucination ve reasoning problemleri kontrollü şekilde test edilebildi.

5. Chunking stratejisi

Doküman doğrudan tek parça olarak embedding modeline verilmedi.

Önce sentence splitting uygulandı. Daha sonra:

2 sentence / chunk

1 sentence overlap

stratejisi kullanıldı.

Örneğin:

Sentence 1

Sentence 2

ilk chunk'ı oluştururken,

Sentence 2

Sentence 3

ikinci chunk'ı oluşturuyordu.

Overlap kullanılmasının nedeni, bir bilginin chunk sınırında parçalanması nedeniyle retrieval sırasında kaybolmasını azaltmaktır.

6. Retrieval sistemi

Her chunk için:

SentenceTransformer("all-MiniLM-L6-v2")

kullanılarak normalize edilmiş embedding üretildi.

Sorgu geldiğinde aynı embedding modeli kullanılarak query embedding oluşturuldu.

Ardından cosine distance üzerinden en yakın chunklar bulundu.

Sistem:

Question

↓

Question embedding

↓

NearestNeighbors

↓

Top-K chunks

şeklinde çalışıyordu.

TOP_K değeri:

3

olarak kullanıldı.

Retrieval için ayrıca bir confidence gate oluşturuldu:

RETRIEVAL_THRESHOLD = 0.50

En iyi retrieval score bu değerin altındaysa sistem LLM'i çalıştırmadan soruyu cevaplanamaz kabul ediyordu.

7. İlk önemli problem: semantic similarity ≠ answerability

İlk retrieval eval setlerinde birçok basit soru başarıyla ayrıldı.

Örneğin:

How long does standard shipping take?

yüksek similarity alırken:

Can I pay with Bitcoin?

düşük similarity alıyordu.

Ancak hard-negative sorular eklendiğinde önemli bir problem ortaya çıktı.

Örneğin:

Do premium members get a student discount?

gibi dokümanda cevabı bulunmayan sorular, premium member kelimeleri nedeniyle yüksek semantic similarity alabiliyordu.

Benzer şekilde:

Can premium members get free express shipping?

sorusu da premium membership ve express shipping bilgilerinin dokümanda ayrı ayrı bulunması nedeniyle yüksek similarity ürettiyordu.

Bu deneylerden çıkan kritik sonuç:

Retrieval similarity, sorunun cevaplanabilir olduğunu garanti etmez.

Dolayısıyla sadece bir similarity threshold kullanarak güvenilir bir RAG sistemi oluşturulamayacağı görüldü.

8. NLI tabanlı verifier

Bu problemi çözmek için pipeline'a bir **Natural Language Inference** katmanı eklendi.

NLI modeli bir premise ve hypothesis olarak üç temel çıktı üretiyor:

Entailment
Neutral
Contradiction

Örneğin:

Premise:

The product must be unused.

Hypothesis:

Used products can be returned.

burada contradiction beklenmektedir.

İlk olarak:

cross-encoder/nli-distilroberta-base

denendi.

Daha sonra daha güçlü:

cross-encoder/nli-deberta-v3-base

modeline geçildi.

DeBERTa özellikle hard-negative örneklerde daha güvenilir sonuç verdi.

9. Whole-answer verification problemi

İlk verifier tasarımında LLM tarafından oluşturulan cevabın tamamı tek hypothesis olarak NLI modeline verildi.

Ancak cevap birden fazla factual claim içerdiğinde problem ortaya çıktı.

Örneğin model şöyle bir cevap üretebiliyordu:

Premium members do not get free express shipping.
Express shipping is available for regular members only.

İlk cümle doğru olsa bile ikinci cümle dokümanda bulunmayan bir bilgi içeriyordu.

Whole-answer NLI bazen doğru claim'i fark edip cevabın tamamını desteklenmiş kabul edebiliyordu.

Bunun üzerine **claim-level verification** geliştirildi.

Yeni mimari:

Generated answer

↓

Claim 1

Claim 2

Claim 3

↓

Her claim ayrı ayrı verify

haline getirildi.

Bir claim bile desteklenmiyorsa tüm cevap reddediliyordu.

10. Evidence selection problemi

Bir sonraki problem NLI modeline verilen evidence tarafında ortaya çıktı.

Başlangıçta verifier tüm chunk üzerinde çalışıyordu.

Örneğin:

Standard shipping usually takes between 3 and 5 business days.

Express shipping is available and normally takes 1 to 2 business days.

gibi iki cümlelik bir premise bazen basit bir factual claim için gereksiz noise yaratıyordu.

Bunun üzerine evidence candidate sistemi geliştirildi.

Her retrieved chunk için:

full chunk

+

individual sentences

evidence candidate olarak oluşturuldu.

Verifier daha sonra claim için en uygun evidence'i seçiyordu.

Bu değişiklik özellikle şu claim'de çok iyi sonuç verdi:

Standard shipping typically takes between 3 and 5 business days.

ve ilgili evidence:

Standard shipping usually takes between 3 and 5 business days.

için yaklaşık:

Entailment ≈ 0.998

elde edildi.

11. Evidence relevance gate

Burada başka bir önemli problem keşfedildi.

NLI modeline yeterince fazla evidence verildiğinde bazen tamamen alakasız bir evidence üzerinde yanlış yüksek entailment oluşabiliyordu.

Örneğin premium express shipping ile ilgili bir claim için warranty cümlesinin yanlışlıkla yüksek entailment aldığı bir deney görüldü.

Bunu engellemek için NLI'dan **önce semantic relevance gate** eklendi.

Threshold:

EVIDENCE_RELEVANCE_THRESHOLD = 0.35

Pipeline böylece:

Claim

↓

Evidence embedding similarity

↓

Relevant evidence?

↓

YES → NLI

NO → reject

haline geldi.

Ardından NLI için:

ENTAILMENT_THRESHOLD = 0.70

kullanıldı.

Bu tasarım iki ayrı soruyu birbirinden ayırmış oldu:

Evidence claim ile ilgili mi?

ve:

Evidence claim'i gerçekten destekliyor mu?

12. Structured output

LLM'in serbest metin üretmesi verifier açısından başka problemlere yol açıyordu.

Örneğin:

Two years

insan açısından anlaşılır bir cevap olsa da NLI açısından iyi bir hypothesis değildir.

Bu nedenle Qwen'den structured JSON üretmesi istendi.

Format:

```
{  
  "answer": "The warranty is two years.",  
  "claims": [  
    "Electronic products include a two-year warranty."  
  ]  
}
```

Böylece kullanıcıya gösterilen answer ile verifier'ın kontrol ettiği factual claims ayrıldı.

Bu değişiklik sistem performansında önemli iyileşme sağladı.

13. Repair pass

Structured claim yine doğrulanamazsa sistem doğrudan cevap vermek yerine LLM'e ikinci bir şans verdi.

Repair prompt içerisinde:

- önceki output,
- verifier feedback,
- desteklenmeyen claim,
- evidence bilgisi

modele tekrar gönderildi.

Amaç modelin claim'i kaynakla uyumlu hale getirmesiydi.

Pipeline:

Initial generation

↓

Verification

↓

Fail

↓

Repair generation

↓

Verification

şeklinde genişletildi.

İlginç biçimde final development setinde repair pass herhangi bir soruyu kurtarmadı:

Repair success count = 0

Ancak held-out testte:

Repair ile kurtarılan = 1

oldu.

Dolayısıyla bu katmanın tamamen gereksiz olmadığı görüldü.

14. RAG V1 sonucu

İlk tam pipeline 22 soruluk development eval üzerinde test edildi.

Sonuç:

Metric	V1
Accuracy	86.36%
Precision	100%
Recall	72.73%
F1	84.21%
TP	8
TN	11
FP	0
FN	3

Burada sistem güvenliydi:

False Positive = 0

ancak bazı cevaplanabilir soruları reddediyordu.

15. RAG V2 — Structured Claims

V2'de structured JSON output ve standalone factual claims eklendi.

Sonuç:

Metric	V2
Accuracy	90.91%
Precision	100%
Recall	81.82%
F1	90.00%
TP	9
TN	11
FP	0
FN	2

V1'e göre belirgin gelişme elde edildi.

Kalan iki false negative:

Can I send my order back after three weeks?

ve:

Can I return a used product after 10 days?

idi.

Burada sistemin iki farklı reasoning problemi olduğu görüldü.

Birincisi:

3 weeks

→ 21 days

→ 21 <= 30

şeklinde aritmetik reasoning gerektiriyordu.

İkincisi ise:

must be unused

bilgisinden:

used product cannot be returned

sonucunun çıkarılmasını gerektiriyordu.

16. V3 deneyi ve regression

V3'te polarity-aware yes/no verification gibi daha karmaşık kurallar eklendi.

Qwen'den:

is_yes_no

polarity

proposition

gibi ek structured alanlar üretmesi istendi.

Ancak kullanılan:

Qwen2.5-0.5B-Instruct

küçük bir model olduğu için prompt karmaşıklığı arttıkça model schema metnini bile output olarak üretmeye başladı.

Örneğin:

"standalone factual claim"

gibi placeholder ifadeler claim olarak üretildi.

V3 sonucu:

Metric	V3
Accuracy	72.73%
Precision	100%
Recall	45.45%
F1	62.50%

Bu ciddi bir regression oldu.

17. V4 deneyi

V4'te deterministic reasoning eklenmeye çalışıldı.

Ancak baseline pipeline'ın diğer parçaları da değiştirildiği için sistem yine regression yaşadı.

Sonuç:

Metric	V4
Accuracy	77.27%
Precision	100%
Recall	54.55%
F1	70.59%

Bu deneylerden önemli bir engineering dersi çıkarıldı:

Bir problemi çözerken çalışan baseline mümkün olduğunca değiştirilmemelidir. Yeni davranış izole bir fallback olarak eklenmelidir.

Bu noktada V2 tekrar gold baseline olarak kabul edildi.

18. Deterministic rescue

Son çözüm, V2'yi değiştirmek yerine yalnızca V2 tamamen başarısız olduktan sonra çalışan dar kapsamlı bir rescue katmanı eklemek oldu.

Akış:

Retrieval

↓

Structured generation

↓

Claim verification

↓

Supported → return

Failed

↓

Repair

↓

Supported → return

Failed

↓

Deterministic rescue

↓

Supported → return

Failed

↓

not_supported

Rescue yalnızca iki spesifik reasoning ailesine uygulanıyordu.

Used-product reasoning

Soru used içeriyorsa şu policy doğrulanıyordu:

The product must be unused and in its original packaging.

Verifier bunu:

Evidence similarity = 1.000

Entailment ≈ 0.993

ile doğruladı.

Sonrasında sistem güvenli şekilde:

No. The return policy requires the product to be unused...

cevabını verebildi.

Duration normalization

Sistem:

three weeks

ifadesini:

$3 \times 7 = 21$ days

olarak normalize etti.

Daha sonra dokümandan:

30-day return window

çıkarıldı ve deterministic comparison yapıldı.

Bu sayede:

$21 \leq 30$

hesaplandı.

19. Final development sonucu

V2 + deterministic rescue ile 22 soruluk development set sonucu:

Metric	Final Dev
Accuracy	100.00%
Precision	100.00%
Recall	100.00%
F1	100.00%
TP	11
TN	11
FP	0
FN	0

Ayrıca:

Repair ile kurtarılan: 0

Deterministic rescue ile kurtarılan: 2

elde edildi.

Ancak bu noktada metodolojik olarak önemli bir karar verildi.

22 soru üzerinde çok sayıda iteration yapıldığı için bu set artık gerçek test seti değildi.

Bu nedenle:

development / tuning set

olarak kabul edildi.

Dolayısıyla %100 sonucu sistemin genel performansı olarak raporlanmadı.

20. Held-out evaluation

Gerçek generalization performansını ölçmek için tamamen yeni 30 soruluk bir held-out set oluşturuldu.

Set:

15 supported

15 unsupported

sorudan oluřtu.

Sorular arasında:

- yeni paraphrase'ler,
- negative questions,
- hard negatives,
- return reasoning,
- warranty questions,
- shipping questions,
- membership questions,
- dokümanda bulunmayan bilgiler

vardı.

Önemli metodolojik nokta řu oldu:

heldout_eval_v1.json **test çalıştırılmadan önce Git'e commit edilip GitHub'a push edildi.**

Böylece test sonuçları görüldükten sonra soruların değıştirilmediğı kayıt altına alınmış oldu.

21. Held-out sonucu

Baseline hiçbir tuning yapılmadan 30 yeni soru üzerinde test edildi.

Sonuç:

Metric	Held-out v1
Accuracy	96.67%
Precision	93.75%
Recall	100.00%
F1	96.77%
True Positive	15
True Negative	14
False Positive	1
False Negative	0

Pipeline statistics:

Repair ile kurtarılan: 1

Deterministic rescue ile kurtarılan: 1

Bu sonuç development setindeki %100 skorundan daha anlamlıdır çünkü sistem bu sorular üzerinde geliştirilmemiřtir.

22. Tek held-out hatasının analizi

Tek false positive:

How much does the premium membership cost?

Doğru karar:

NOT_SUPPORTED

çünkü bilgi kaynağında premium membership fiyatı bulunmamaktadır.

Ancak sistem:

Free standard shipping on all orders.

cevabını üretti.

Verifier açısından bu claim gerçekten doğrudu:

Evidence:

Premium members receive free standard shipping on all orders.

Evidence similarity: 0.6939

Entailment: 0.9967

Dolayısıyla verifier'ın yaptığı işlem teknik olarak hatalı değildi.

Gerçek problem:

Question:

Premium membership kaç para?

Generated claim:

Premium members get free shipping.

olmasıydı.

Claim dokümana dayanıyordu fakat **kullanıcının sorusunu cevaplamıyordu.**

Bu nedenle şu anki sistemin bilinen ana limitation'ı:

Question-Answer / Question-Claim relevance kontrolünün ayrı bir katman olarak bulunmaması.

Mevcut verifier şu soruya çok iyi cevap veriyor:

Bu claim doküman tarafından destekleniyor mu?

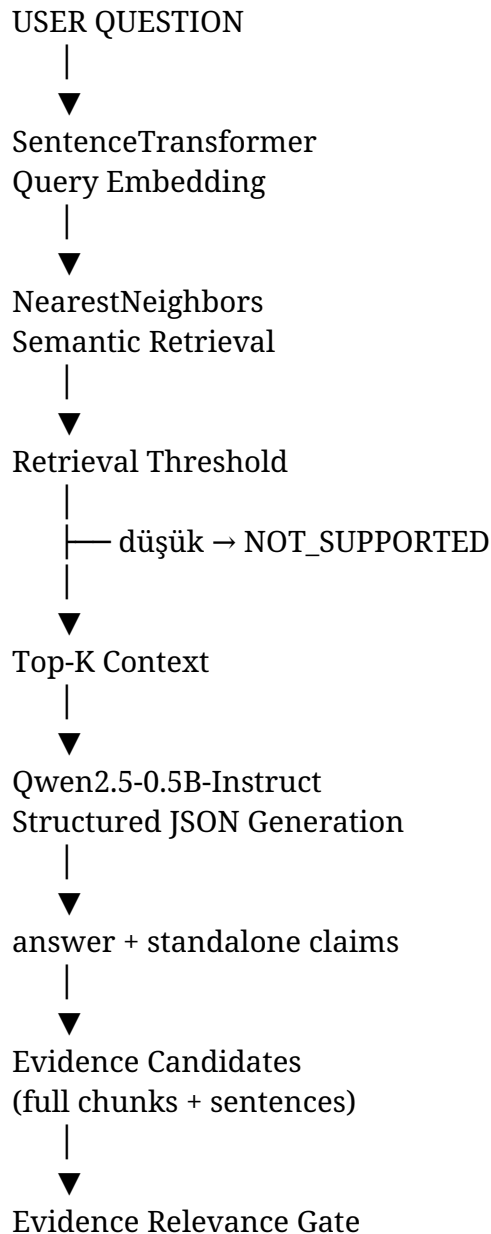
Ancak henüz ayrı olarak şunu değerlendirmiyor:

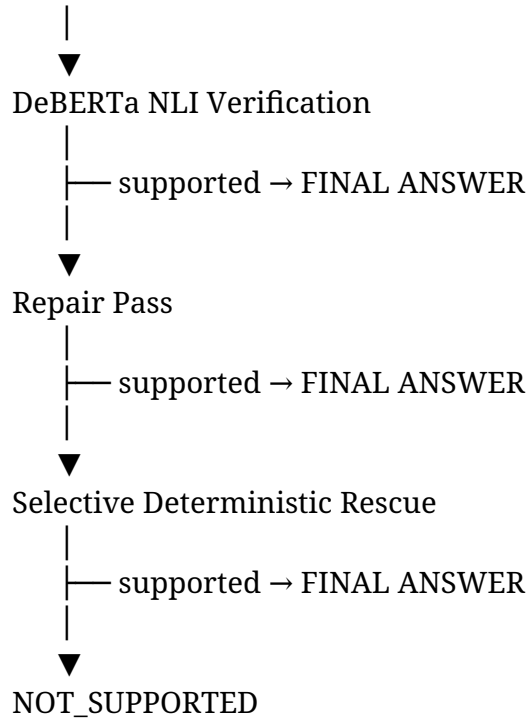
Bu claim kullanıcının sorduğu şeyi gerçekten cevaplıyor mu?

Bu, gelecekte araştırılabilecek bir geliştirme alanı olarak bırakıldı. Baseline sırf bu tek hata nedeniyle değiştirilmedi.

23. Final mimari

Mevcut çalışan sistemin genel mimarisi:





Bu yapı klasik basit bir RAG sisteminden daha ileri durumdadır.

Çünkü sadece:

retrieve → generate

yapmamakta; ayrıca:

retrieve

→ generate

→ decompose

→ relevance filter

→ verify

→ repair

→ deterministic reasoning

→ final decision

pipeline'ını uygulamaktadır.

24. Final baseline

Ana çalışan implementation:

eval_full_rag_v2.py

Development sonuç dosyası:

full_rag_eval_v2_rescue_results.json

Held-out evaluator:

eval_heldout_v1.py

Held-out dataset:

heldout_eval_v1.json

Held-out sonuç dosyası:

heldout_eval_v1_results.json

25. Git ve reproducibility

Proje Git version control altına alındı.

.gitignore içerisine özellikle:

```
.venv/  
__pycache__/  
*.pyc  
.DS_Store  
review_embeddings.npy
```

eklendi.

İlk çalışan baseline commit'i oluşturuldu:

74fcd9d
RAG baseline v1.0

Ardından annotated Git tag oluşturuldu:

rag-baseline-v1.0

Bu tag GitHub'a da push edildi.

Bunun anlamı şudur:

Gelecekte proje ne kadar değişirse değişsin, çalışan baseline'ın tam kaynak koduna geri dönmek mümkündür.

Repository için README oluşturularak ana implementation ve eval sonuçları belgelendi.

Held-out dataset de evaluation yapılmadan önce commit edildi.

Bu nedenle projenin deney süreci Git geçmişinden takip edilebilir durumdadır.

26. Deneylerden çıkarılan temel mühendislik dersleri

Bu projede yalnızca çalışan bir RAG sistemi değil, önemli AI engineering prensipleri de ortaya çıktı:

1. **Semantic similarity answerability değildir.** Bir metnin soruyla benzer olması, sorunun cevabını içerdiği anlamına gelmez.
 2. **LLM generation tek başına güvenilir değildir.** Küçük lokal modeller doğru context verilse bile yanlış veya alakasız cevap üretebilir.
 3. **NLI verifier hallucination kontrolünde faydalıdır ancak kusursuz değildir.**
 4. **Evidence relevance, entailment kontrolünden ayrı yapılmalıdır.**
 5. **Whole-answer verification yerine claim-level verification daha güvenlidir.**
 6. **Structured claims, kısa veya conversational cevaplara göre çok daha kolay doğrulanır.**
 7. **Daha karmaşık prompt her zaman daha iyi sonuç vermez.** Özellikle küçük modeller schema complexity arttığında ciddi şekilde bozulabilir.
 8. **Regression testing zorunludur.** V3 ve V4 deneylerinde yeni özellikler eski çalışan davranışları bozmuştur.
 9. **İyi bir baseline dondurulmalıdır.** Yeni özellikler baseline'ın içine kontrolsüz şekilde eklenmemelidir.
 10. **Development ve held-out test sonuçları birbirinden ayrılmalıdır.** Development setinde %100 görmek gerçek generalization performansının %100 olduğu anlamına gelmez.
-

27. Mevcut sistemin durumu

Şu anda proje teknik olarak başarılı bir **local grounded RAG baseline** durumundadır.

Development performansı:

Accuracy: 100%

Precision: 100%

Recall: 100%

F1: 100%

Bağımsız held-out performansı:

Accuracy: 96.67%

Precision: 93.75%

Recall: 100%
F1: 96.77%

En önemlisi:

False Negative = 0

olmuştur.

Yani held-out sette cevaplanabilir 15 sorunun tamamı sistem tarafından cevaplanabilir olarak bulunmuştur.

Tek hata, factual hallucination yerine **answer relevance failure** olmuştur.

28. Projenin şu an ulaştığı seviye

Başlangıçta proje semantic similarity deneylerinden ibaretti.

Bugün ise elimizde:

Local LLM

+

Semantic Retriever

+

Chunking

+

Vector Search

+

Retrieval Confidence Gate

+

Structured Generation

+

Claim Extraction

+

Evidence Relevance

+

NLI Grounding

+

Repair Pipeline

+

Deterministic Reasoning

+

Development Eval

+

Held-Out Eval

+

Regression Analysis

+

Git Versioning

+

Reproducible Evaluation Results

bulunuyor.

Dolayısıyla proje artık yalnızca “RAG nasıl yapılır?” deneyi olmaktan çıkıp, **ölçülebilen, versiyonlanan ve failure analysis yapılan bir AI engineering projesi** haline gelmiş durumda.

En anlamlı mevcut sonuç şu şekilde özetlenebilir:

Tamamen lokal ve ücretsiz modellerle geliştirilen RAG pipeline, önceden sabitlenmiş 30 soruluk bağımsız held-out evaluation setinde %96.67 accuracy ve %96.77 F1 elde etti. Sistem semantic retrieval, structured claim generation, evidence relevance filtering, NLI tabanlı grounding verification ve selective deterministic reasoning katmanlarını bir arada kullanmaktadır.